Python: comunicazione seriale con arduino

Obiettivo

L'obiettivo di questa attività è imparare a comunicare tra Python e Arduino attraverso la porta seriale, esplorando entrambe le direzioni di comunicazione.

Attenzione - Ambiente Virtuale Anaconda

> **IMPORTANTE**: Per questa attività lavorerete con **Anaconda** e un ambiente virtuale.
>
> Prima di iniziare, dovrete **installare le librerie necessarie** nell'ambiente virtuale in cui lavorate. Non tutte le librerie sono già presenti di default!
>
> **Suggerimento**: Testate il vostro codice e verificate quali librerie mancano. Imparate a riconoscere gli errori di importazione e a risolverli installando i pacchetti corretti.

Comandi Base per la Comunicazione Seriale

In Python

Per configurare e utilizzare la comunicazione seriale in Python, ecco i comandi essenziali:

```
import serial

# Configurazione della connessione seriale
connessione = serial.Serial('COM3', 9600)  # Sostituire COM3 con la porta corretta

# Leggere una riga dalla seriale (fino a '\n')
dato = connessione.readline()
# Esempio: b'25.3\n' -> restituisce l'intera riga come bytes

# Leggere un singolo byte
byte_singolo = connessione.read()
# Esempio: legge solo 1 byte alla volta

# Leggere un numero specifico di byte
dati = connessione.read(5)
# Esempio: legge esattamente 5 byte

# Verificare quanti byte sono disponibili nel buffer
if connessione.in_waiting > 0:
```

```
    dati = connessione.read(connessione.in_waiting)
    # Legge tutti i byte disponibili

# Scrivere sulla seriale
connessione.write(b'messaggio')  # Il prefisso b indica byte
connessione.write('ON\n')  # Questo non è corretto, se avete un messaggio in variabile dovete
                           # la soluzione per farlo funzionare
                           # è stato scritto così per non fornirvi la soluzione
```

Differenze tra i metodi di lettura in Python:

Metodo	Descrizione	Quando usarlo
`readline()`	Legge fino al carattere `\n`	Quando Arduino invia righe complete con `Serial.println()`
`read()`	Legge 1 byte	Quando serve leggere carattere per carattere
`read(n)`	Legge esattamente n byte	Quando si conosce la lunghezza esatta dei dati
`in_waiting`	Conta byte disponibili	Per verificare se ci sono dati prima di leggere

In Arduino

```
// Leggere un singolo byte/carattere dalla seriale
if (Serial.available() > 0) {
    char dato = Serial.read();
    // Legge un solo carattere alla volta
}

// Leggere più byte in un array
byte buffer[10];
if (Serial.available() > 0) {
    int numBytes = Serial.readBytes(buffer, 10);
    // Legge fino a 10 byte e li salva in buffer
    // Restituisce il numero di byte effettivamente letti
}

// Leggere una stringa fino a un carattere terminatore
String messaggio = "";
if (Serial.available() > 0) {
    messaggio = Serial.readStringUntil('\n');
    // Legge tutti i caratteri fino a trovare '\n'
    // Utile per leggere comandi completi inviati da Python
}

// Scrivere sulla seriale
```

```
Serial.println("messaggio");  // Invia messaggio + '\n'
Serial.print("messaggio");    // Invia solo il messaggio senza '\n'
```

Differenze tra i metodi di lettura in Arduino:

Metodo	Descrizione	Quando usarlo
`Serial.read()`	Legge 1 byte	Per comandi singoli (es. '1', '0')
`Serial.readBytes(buffer, length)`	Legge n byte in un array	Per dati binari o lunghezza fissa
`Serial.readStringUntil(char)`	Legge fino a un carattere	Per comandi testuali completi (es. "ACCENDI\n")
`Serial.available()`	Conta byte disponibili	Per verificare se ci sono dati da leggere

Attività 1: Arduino Scrive → Python Legge

Obiettivo

Creare un programma Arduino che invia dati sulla seriale e un programma Python che li legge e li visualizza.

Fase 1.1: Programma Arduino

Scrivere un programma Arduino che:

- Invia un messaggio o un valore numerico sulla seriale ogni secondo

Fase 1.2: Programma Python

Scrivere un programma Python che:

- Si connette alla porta seriale corretta
- Legge i dati inviati da Arduino
- Stampa i dati ricevuti sulla console

Suggerimenti:

- Utilizzare `connessione.readline()` per leggere i dati

Verifica

Eseguire il programma Python da **Anaconda Prompt** e verificare che i dati inviati da Arduino vengano visualizzati correttamente sulla console.

Attività 2: Python Scrive → Arduino Legge

Obiettivo

Creare un programma Python che invia comandi sulla seriale e un programma Arduino che li riceve e controlla il LED integrato.

Fase 2.1: Programma Arduino

Scrivere un programma Arduino che:

- Legge comandi dalla seriale
- Accende il LED integrato (pin 13 o `LED_BUILTIN`) quando riceve un comando specifico (es. '1' o 'ON')
- Spegne il LED quando riceve un altro comando (es. '0' o 'OFF')

Fase 2.2: Programma Python

Scrivere un programma Python che:

- Si connette alla porta seriale
- Chiede all'utente quale comando eseguire (accendere o spegnere il LED)
- Invia il comando ad Arduino utilizzando `connessione.write()`

Suggerimenti:

- Ricordarsi di aprire la connessione seriale all'inizio del programma

Verifica

Eseguire il programma Python da **Anaconda Prompt**:

1. Dovrebbe comparire una richiesta di inserimento del comando
2. Inserendo il comando appropriato, il LED su Arduino dovrebbe accendersi o spegnersi
3. Testare più volte per verificare il corretto funzionamento

Note Finali

- **Porta seriale**: Verificare quale porta COM è assegnata ad Arduino (controllare in Gestione Dispositivi su Windows o nell'IDE Arduino)
- **Baud rate**: Assicurarsi che il baud rate sia lo stesso in Arduino e Python (es. 9600)
- **Chiusura connessione**: Ricordarsi di chiudere la connessione seriale in Python con `connessione.close()` quando il programma termina
- **Debugging**: Se qualcosa non funziona, utilizzare il Monitor Seriale dell'IDE Arduino per verificare cosa viene inviato/ricevuto

Consegna

Per ciascuna attività, consegnare:

1. Il codice Arduino (file `.ino`)
2. Il codice Python (file `.py`)
3. Una breve relazione che descrive:
    - Le librerie installate nell'ambiente Anaconda
    - Eventuali problemi riscontrati e come sono stati risolti
    - Screenshot del funzionamento dei programmi